

BFS Water Jug Problem – Python

4 L Jug + 3 L Jug | Goal = 2 L | Starting State = (0, 0)

1. Python Program

```
from collections import deque

# Jug capacities and goal
CAP_A = 4
CAP_B = 3
GOAL = 2

def print_state(state):
    print(f"Jug A = {state[0]} L, Jug B = {state[1]} L")

# Generate all possible next states
def get_neighbors(state):
    a, b = state
    neighbors = []

    # 1. Fill Jug A
    if a < CAP_A:
        neighbors.append(((CAP_A, b), "Fill Jug A"))

    # 2. Fill Jug B
    if b < CAP_B:
        neighbors.append(((a, CAP_B), "Fill Jug B"))

    # 3. Empty Jug A
    if a > 0:
        neighbors.append(((0, b), "Empty Jug A"))

    # 4. Empty Jug B
    if b > 0:
        neighbors.append(((a, 0), "Empty Jug B"))

    # 5. Pour Jug A -> Jug B
    amount = min(a, CAP_B - b)
    if amount > 0:
        neighbors.append(
            ((a - amount, b + amount),
             "Pour Jug A -> Jug B")
        )

    # 6. Pour Jug B -> Jug A
    amount = min(b, CAP_A - a)
    if amount > 0:
        neighbors.append(
            ((a + amount, b - amount),
             "Pour Jug B -> Jug A")
        )

    return neighbors

# BFS Algorithm
def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue

        visited.add(state)

        # Check goal
        if state[0] == GOAL or state[1] == GOAL:
            return path + [(state, "Goal Reached")]

        # Generate neighbors
        for neighbor, action in get_neighbors(state):
            if neighbor not in visited:
                queue.append(
                    (neighbor, path + [(neighbor, action)])
                )
```

```

        return None

# Starting state
start = (0, 0)

# Run BFS
solution = bfs(start)

# Print solution
if solution:
    print("Solution found in", len(solution) - 1, "moves:")
    print("\nInitial State:")
    print_state(start)

    for state, action in solution:
        print(action)
        print_state(state)
else:
    print("No solution found.")

```

2. Output

Solution found in 4 moves:

```

Initial State:
Jug A = 0 L, Jug B = 0 L
Fill Jug B
Jug A = 0 L, Jug B = 3 L
Pour Jug B -> Jug A
Jug A = 3 L, Jug B = 0 L
Fill Jug B
Jug A = 3 L, Jug B = 3 L
Pour Jug B -> Jug A
Jug A = 4 L, Jug B = 2 L
Goal Reached
Jug A = 4 L, Jug B = 2 L

```

Result: BFS finds the shortest solution in 4 moves, reaching the state (4, 2).